

Staj Final Raporu — Polyglot / Steganaliz Servisi

Proje: X (Twitter) Platformunda Görsel Arkasına Saklanmış Video ve Veri Tespiti (Polyglot / Steganaliz Servisi)

Süre: 20 iş günü (4 hafta), Ağustos-Eylül 2026

Kapsam: Bu proje yalnızca eğitim ve savunma (defensive security) amaçlıdır. Gerçek kullanıcı verisi toplanmadı veya kazınmadı (scraping yok); tüm test dosyaları proje kapsamında sentetik olarak üretildi.

1. Özet

Bu staj kapsamında, sosyal medya platformlarında paylaşılan görsel dosyaların (PNG/JPEG) arkasına gizlenmiş video verisini ("polyglot" dosyalar) tespit eden, ayıklayan ve bir web arayüzü üzerinden raporlayan uçtan uca bir sistem geliştirildi. Proje siber güvenlik (dosya formatı analizi, steganaliz), bilgisayar görüşü (piksel/bayt düzeyinde anomali tespiti) ve full-stack web geliştirmeyi (FastAPI backend + vanilla JS frontend) tek bir çalışmada birleştirdi.

20 günlük plan, 4 haftalık aşamada tamamlandı:

- Hafta 1 (Gün 1-5):** Dosya format mimarisi (PNG/JPEG/MP4 binary yapısı), sentetik polyglot üretici, EOF-ötesi trailer tespiti, Shannon entropy analizi.
- Hafta 2 (Gün 6-10):** Teorik/gerçek boyut sapma analizi, LSB/DCT gürültü analizi, extraction (gizli videoyu ayıklama), video meta verisi, çok senaryolu test başarımları ölçümü.
- Hafta 3 (Gün 11-15):** Steganaliz motorunun FastAPI REST servisine dönüştürülmesi: dosya yükleme, asenkron pipeline entegrasyonu, `threat_score / analysis_summary` şeması, hata yönetimi.
- Hafta 4 (Gün 16-20):** Sürükle-bırak web arayüzü, frontend-backend entegrasyonu, uçtan uca doğrulama testleri, dokümantasyon ve bu final raporu.

Nihai durum: Planın 20 gününün tamamı tamamlandı ve bağımsız olarak yeniden doğrulandı (bkz. Bölüm 5). Sistem, sentetik test setinde %0 yanlış pozitif / %0 yanlış negatif oranıyla çalışıyor.

2. Yöntem

2.1 Tehdit Modeli

Projenin ana tehdit modeli **trailer-append polyglot**: meşru bir görselin gerçek dosya sonu işaretinden (PNG IEND + CRC, JPEG FFD9) sonra, o formatın çoğu görüntüleyicisi/tarayıcısı tarafından yok sayılan ham bir video akışının (MP4/ ftyp) doğrudan eklenmesidir. Bu, çoğu platformun görsel önizlemesini bozmadan ek veri taşımanın en basit yoludur.

2.2 Tespit Mimarisi

Üç sinyal bağımsız olarak hesaplanıp ağırlıklı şekilde birleştiriliyor:

Sinyal	Ne ölçer	Ağırlık	Rol
Trailer/imza tespiti (detect_trailer.py)	EOF sonrası bilinen video imzası (ftyp / moov /RIFF/EBML) var mı	60	Birincil, en güvenilir sinyal
Shannon entropy farkı (entropy.py)	Görsel bölgesi ile trailer bölgesi arasındaki blok-bazlı entropy sıçraması	25	Doğrulayıcı/tamamlayıcı
Teorik/gerçek boyut sapması (size_analysis.py)	Çözünürlük+renk derinliğinden beklenen boyutla gerçek boyut farkı	15	Doğrulayıcı/tamamlayıcı

Bu üçü, backend/app/pipeline.py::compute_threat_score içinde 0-100 aralığında tek bir threat_score e dönüştürülüyor. Trailer tespiti tek başına en yüksek ağırlığı taşıyor çünkü Hafta 2'de (Gün 10) diğer iki sinyalin tek başına yanlış pozitif/negatif üretebildiği gözlemlendi (bkz. Bölüm 3). Dosya polyglot ise (extract.py), trailer'ın başlangıç offset'inden bölünüp gizli video bağımsız bir .mp4 olarak kaydediliyor ve meta verisi (video_metadata.py : kare sayısı, süre, çözünürlük, codec) çıkarılıyor.

Tamamlayıcı olarak LSB (lsb_analysis.py) ve DCT (dct_analysis.py) gürültü analizleri de geliştirildi; bunlar trailer-append polyglot'u yakalamaz (beklenen davranış, bkz. Bölüm 3) ama ileride LSB tabanlı steganografi tehdit modeline genişletilme ihtimaline karşı ayrı bir sinyal olarak konumlandırıldı.

2.3 Sistem Mimarisi

```

Tarayıcı (frontend/app.js)
  | fetch POST, multipart/form-data
  ▼
FastAPI /api/v1/analyze (backend/app/main.py)
  | magic bytes + boyut doğrulaması, backend/tmp/ altına kaydetme
  | asyncio.to_thread(...) - event loop'u bloklamadan ayrı thread'de çalıştırma
  ▼
pipeline.run_pipeline (backend/app/pipeline.py)
  | detect_trailer → size_analysis → entropy → (polyglot ise) extract → video_metadata

```

```

▼
compute_threat_score + build_analysis_summary

▼
AnalyzeResponse (JSON) → frontend'de risk skoru/özet gösterimi
└─ extracted_video_url → StaticFiles ("/media") → <video> player

```

`scripts/` altındaki tüm analiz modülleri backend olmadan da bağımsız CLI araçları olarak çalışabiliyor; `backend/`, bunları HTTP üzerinden sarmalayan ince bir katman.

3. Karşılaşılan Zorluklar

- **JPEG'de görsel/video ayrımının PNG'ye göre daha zor olması (Gün 5):** JPEG'in kendisi DCT tabanlı sıkıştırma nedeniyle zaten yüksek entropili veri ürettiğinden, entropy sinyali tek başına JPEG polyglot'larda daha az belirgin. Bu, entropy'nin neden yalnızca destekleyici bir sinyal olarak (trailer tespitinin yanında) konumlandırıldığını doğruladı.
- **Yeniden sıkıştırmanın trailer verisini bozabilmesi (Gün 10):** Bir görsel optimizasyon aracıyla yeniden sıkıştırılan polyglot dosyalarda, trailer'ın bir kısmı/tamamı kaybolabiliyor. Bu bilinçli olarak bir sınırlama kabul edildi — aynı zamanda istenmeyen yükün kendiliğinden etkisizleşmesi bakımından olumlu bir yan etki olarak değerlendirildi.
- **size_analysis'in LSB-stego dosyalarında yanlış pozitif verebilmesi (Gün 10/14):** Boyut sapması sinyali, trailer içermeyen ama farklı bir gizleme tekniği (LSB) kullanan dosyalarda da "şüpheli" işareti verebiliyor. Bu nedenle bu sinyal API katmanında hiçbir zaman tek başına karar mercii yapılmadı; yalnızca trailer tespitini destekleyen, düşük ağırlıklı bir bileşen olarak `threat_score` e katıldı.
- **LSB/DCT analizlerinin trailer-append polyglot'u yakalayamaması (Gün 7):** Bu beklenen ve doğrulanmış bir sonuç — LSB/DCT, piksel verisini değiştirmeyen trailer-append tekniğine karşı görünürde bir iz bırakmıyor. Bu iki araç bilinçli olarak ana tespit mekanizması yerine tamamlayıcı sinyal konumunda tutuldu.
- **Canlı tarayıcı otomasyonunun bulunmaması (Gün 16-20):** Geliştirme ortamında `claude-in-chrome` bağlı olmadığından, frontend'in sürükle- bırak/tıklama etkileşimi görsel olarak otomatik doğrulanamadı. Bunun yerine, frontend'in attığı isteğin birebiri HTTP düzeyinde (aynı uç nokta, aynı `Origin` davranışı) test edilerek işlevsel olarak eşdeğer bir doğrulama yapıldı; her aşamada kullanıcının kendi tarayıcısında son bir görsel teyit yapması önerisi tutarlı şekilde not edildi.
- **scripts/ 'i paketleştirmeden API'ye bağlama kararı (Gün 13):** `scripts/` modülleri birbirini bare isimle import ettiğinden (paket değil), bunları API katmanından kullanmak için ya paketleştirmek (regresyon riski) ya da `sys.path` bootstrap'i (mevcut CLI davranışını koruma) arasında seçim yapıldı. İkincisi seçildi — test edilmiş bir arayüzü bozmadan yeni bir katman eklemek, "daha temiz" görünen ama regresyon riski taşıyan bir refactor'a tercih edildi.

4. Sonuçlar

4.1 Test Başarımı

Gün 10'da CLI pipeline için, Gün 18'de gerçek HTTP API + frontend entegrasyonu üzerinden ölçülen sonuçlar:

Test seti	Dosya sayısı	polyglot_status doğruluğu	FP	FN
Gün 10 (CLI, 4 senaryo)	çeşitli PNG/JPEG/yeniden sıkıştırılmış/temiz	✓	~%0	~%0
Gün 18 (HTTP API + web arayüzü, 12 dosya: 6 temiz + 6 polyglot)	12/12 beklenen sonuç	✓	0/6 (%0)	0/6 (%0)

Ayrıntılı tablo: <docs/test-sonuclari.md>.

4.2 Nihai Bileşen Durumu

Bileşen	Durum
Format analizi ve polyglot üretici (<code>scripts/</code>)	✓ Hazır
Steganaliz motoru (trailer, entropy, boyut sapması, LSB/DCT, extraction)	✓ Hazır
FastAPI backend (<code>backend/</code> , CORS dahil)	✓ Hazır
Web arayüzü (<code>frontend/</code> , backend'e canlı bağlı)	✓ Hazır, uçtan uca doğrulandı
Dokümantasyon (README, docstring'ler, <code>docs/</code> indeksi)	✓ Hazır

4.3 Gün 20 Bağımsız Doğrulaması

Bu raporun hazırlanması sırasında Gün 1-19'un tamamı yeniden test edildi (ortam kontrolü, CLI pipeline, extraction, canlı `uvicorn` sunucusuna karşı API testleri) — tüm sonuçlar önceki günlerin raporlarıyla birebir eşleşti, hiçbir hata veya eksik bulunmadı. Ayrıntı için: <docs/gun20-demo-ve-kapanis-raporu.md>.

4.4 Kapsam Dışı Bırakılan / Gelecek Geliştirmeler

Aşağıdakiler bilinçli olarak 20 günlük planın kapsamı dışında tutuldu; proje tamamlandıktan sonra istenirse ayrı ayrı ele alınabilir (ayrıntı: <PLAN.md>):

- Otomatik `pytest` test paketi (şu ana kadar tüm testler manuel/Gün 10 ve Gün 18'de dokümante edilmiş şekilde yapıldı)
- README'ye ekran görüntüsü/GIF, görsel mimari diyagramı
- Küçük frontend UI cılası, `LICENSE` dosyası

5. Kapanış

20 günlük plan, planlanan tüm hedeflere ulaşarak tamamlandı: sosyal medya görsellerinde gizlenmiş video/veriyi tespit eden, ayıklayan ve bir web arayüzü üzerinden raporlayan, dışarıdan sorgulanabilir tam fonksiyonel bir Steganaliz Servisi. Detaylı günlük/haftalık raporlar için [docs/README.md](#) indeksine bakılabilir.